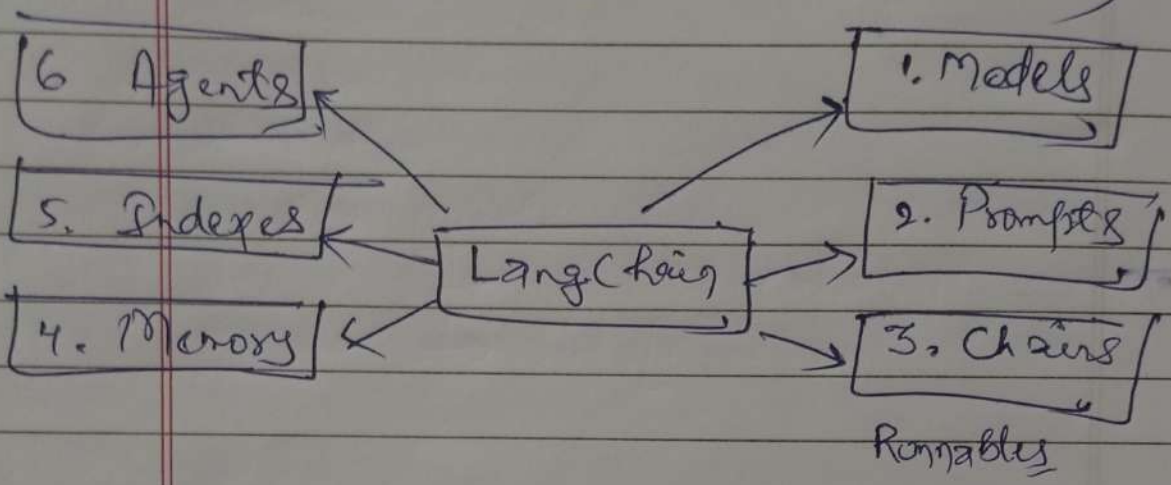
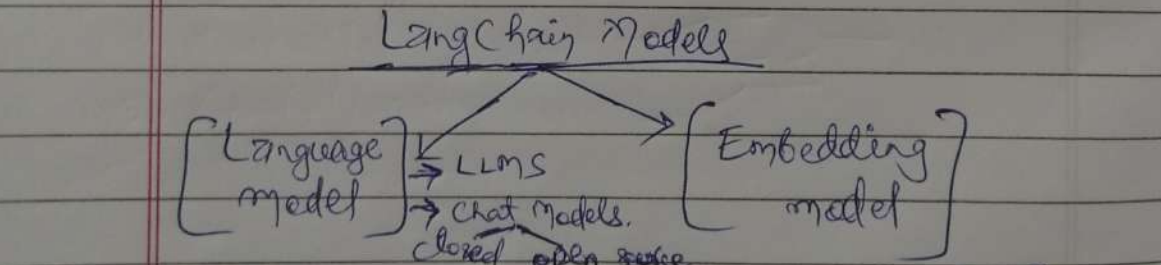
LangChain Components

LangChain is an open-source framework for developing applications powered by large language models (LLMs).



1. Models

In LangChain, 'models' are the core interfaces through which you interact with AI models.



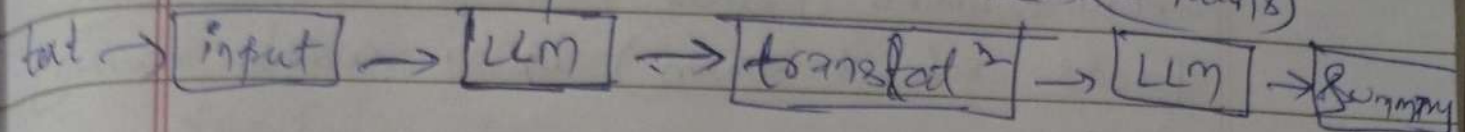
2. Prompts

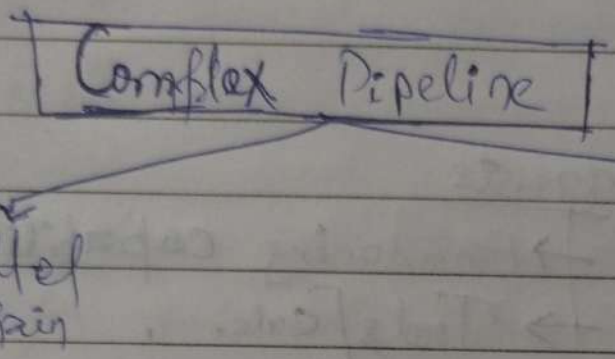
LLM → input → prompt

- Dynamic & Reusable Prompts
- Role-Based Prompts
- Few Shot Prompting

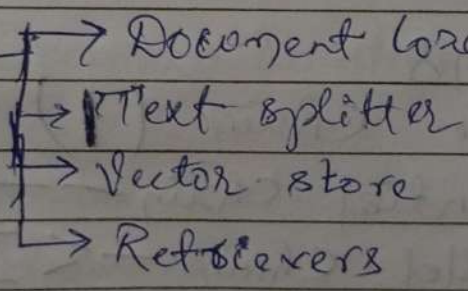
4. Chains

LLMs → Pipelines → (chains)





A. Indexes connect your application to external knowledge — such as PDFs, websites or ~~data~~ databases.



5. Memory

LLM' Api calls are stateless.

- ConversationBufferMemory : Stores a transcript of recent messages. Great for short chats but can grow large quickly.
- ConversationBufferWindowMemory : Only keeps the last N interactions to avoid excessive token usage.
- Summarize-Based Memory : Periodically summarizes older chat segments to keep a condensed memory footprint.

- Custom Memory : For advanced use cases you can store specialized state (eg. the user's preferences or key facts about them) in a custom memory class.

6. Agents

AI agents
→ Reasoning capability
→ Tools [Calc. ,

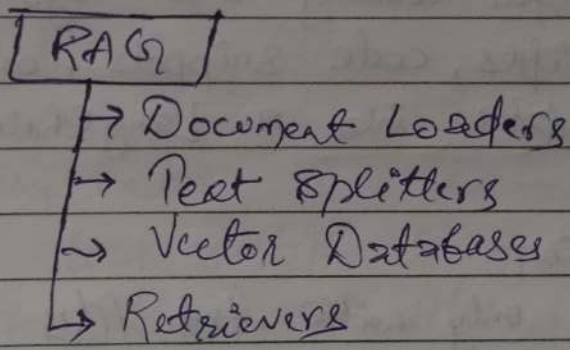
Chains

- Simple Chain (1)
- Sequential Chain — — — — —
- Parallel Chain —<—>—
- Conditional Chain —<—>—

RAG is a technique that combines information retrieval with language generation, where a model retrieves relevant documents from a knowledge base and then uses them as context to generate accurate and grounded responses.

Benefits of using RAG

1. Use of up-to-date information
2. Better privacy
3. No limit of document size.



* Document Loaders → Main Concepts
  - TextLoader
  - PyPDFLoader
  - WebBaseLoader
  - CSVLoader

→ Document Loaders are components in LangChain used to load data from various sources into a standardized format (usually as Document objects), which can then be used for chunking, embedding, retrieval, and generation.

Document(
page_content = "Title - - - - - "
metadata = {"source": "filename.pdf", ...}
)

* TextLoader

TextLoader is a simple and commonly used document loader in LangChain that reads plain text (.txt) files and converts them into LangChain Document objects.

Use Case

- Ideal for loading chat logs, scraped text, transcripts, code snippets, or any plain text data into a LangChain pipeline.

Limitation

- Works only with .txt files

* from langchain_community.document_loaders import TextLoader
loader = TextLoader('cricket.txt', encoding='utf-8')
docs = loader.load()
[[ ], [ ], [ ]] lists

* PyPDFLoader

PyPDFLoader is a document loader in LangChain used to load content from PDF files and convert each page into a Document object.

25 page pdf → 25 document

Limitations:

- It uses the PyPDF library under the hood - not great with scanned PDFs or complex layouts.

Use Case	Recommended Loader
• Simple, clean PDFs	PyPDFLoader
• PDFs with tables/columns	PDFPlumberLoader
• Scanned/image PDFs	UnstructuredPDFLoader or AmazonTextractPDFLoader
• Need layout and image data	PyMuPDFLoader
• Want best structure extraction	UnstructuredPDFLoader

* DirectoryLoader is a document loader that lets you load multiple documents from a directory (folder) of files.

Text Splitters in LangChain

Text Splitting is the process of breaking large chunks of text (like articles, PDFs, HTML pages or books) into smaller, manageable pieces (chunks) that an LLM can handle effectively.

1. Length Based Text Splitting
2. Text-Structured Based (Recursive)
3. Document-Structured Based
4. Semantic Meaning Based.

Vectors Store.

Problems
- Generate embedding vectors
- storage
- Semantic search (intelligent)

- A vector store is a system designed to store and retrieve data represented as numerical vectors.

* Key features.
1. Storage - Ensures that vectors and their associated metadata are retained, whether in-memory for quick lookups or on disk for

durability and large-scale use.

2. Similarity Search - Helps retrieve the vectors most similar to a query vector.

3. Indexing - Provide a data structure or method that enables fast ~~sim~~ similarity searches on high-dimensional vectors (e.g., approximate nearest neighbor lookups).

4. CRUD Operations - Manage the lifecycle of data - adding new vectors, reading them, updating existing entries, removing outdated vectors.

* Use-cases.
1. Semantic Search
2. RAG
3. Recommender Systems
4. Image/Multimodal search.

Vector Store Vs Vector Database

→ Vector Store

- Typically refers to a lightweight library or service that focuses on storing vectors (embeddings) and performing similarity search.
- May not include many traditional database features like transactions, rich query languages, or role-based access control.

- Ideal for prototyping, smaller-scale applications.
- Ex: FAISS (where you store vectors and can query them by similarity, but you handle persistence and scaling separately).

→ Vector Database

- A full-fledged database system designed to store and query vectors.
- Offers additional "database-like" features:
  - Distributed architecture for horizontal scaling.
  - Durability and persistence (replication, backup/restore)
  - Metadata handling (schemas, filters)
  - Potential of ACID or near-ACID guarantees.
  - Authentication/authorization and more advanced security.
- Geared for prod^2 env. with significant scaling, large datasets
- Ex: Milvus, Qdrant, Weaviate.

* A vector database is effectively a vector store with extra database features (e.g., clustering, scaling, security, metadata filtering, and durability)

Vector Stores in Langchain

- Supported Stores: Langchain integrates with multiple vector stores (FAISS, Pinecone, Chroma, Qdrant, Weaviate. etc.) giving you flexibility in scale, features, and deployment.

- Common Interface: A uniform Vector Store API lets you swap out one backend (e.g. FAISS) for another (e.g., Pinecone) with minimal code changes.

- Metadata Handling: Most vector stores in LangChain allow you to attach metadata (e.g., timestamps, authors) to each document, enabling filter-based retrieval.

Chroma Vector Store

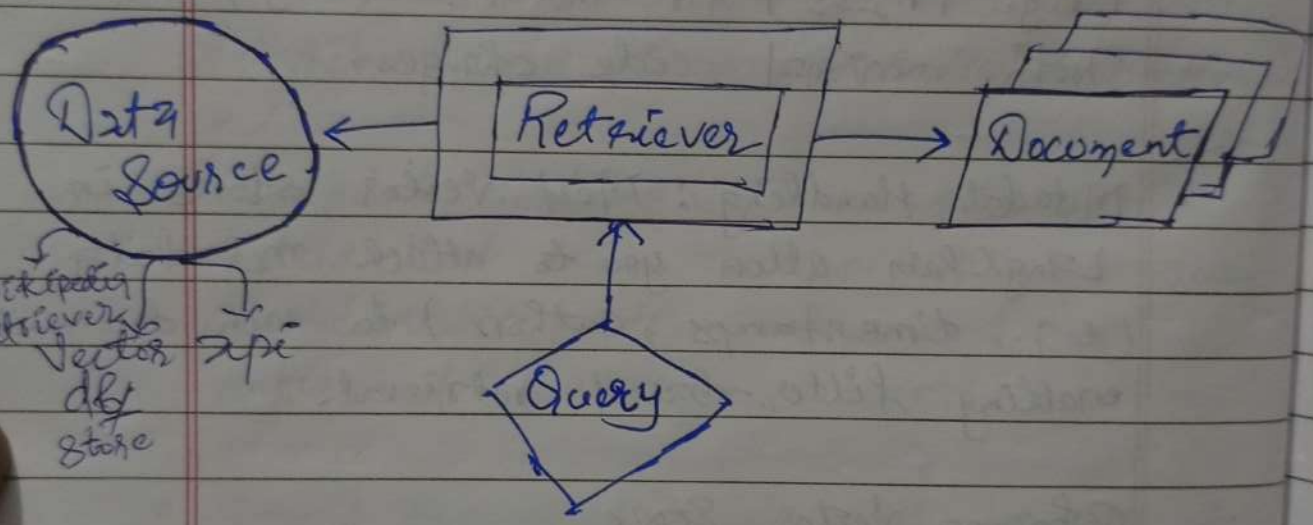
Chroma is a lightweight, open-source vector database that is especially friendly for local development and small to medium-scale production needs.

What are Retrievers

→ A Retriever is a component in LangChain that fetches relevant documents from a data source in response to a user's query.

→ There are multiple type of retrievers.

→ All retrievers in LangChain are runnable



Wikipedia Retriever. is a retriever that queries the Wikipedia API to fetch relevant content for a given query.

How It Works.

1. You give it a query (e.g., "Albert Einstein")
2. It sends the query to Wikipedia's API
3. It retrieves the most relevant articles.
4. It returns them as LangChain Document objects.

Vector Store Retriever

A Vector Store Retriever in LangChain is the most common type of retriever that lets you search and fetch documents from a vector store based on semantic similarity using vector embeddings.

- How It works

1. You store your documents in a vector store (like FAISS, Chroma, Weaviate)
2. Each document is converted into a dense vector using an embedding model.
3. When the user enters a query:
   - Its also turned into a vector.
   - The retriever compares the query vector with the stored vectors.
   - It retrieves the top-k most similar ones.

TO maximum Marginal Relevance (MMR)

"How can we pick results that are not only relevant to the query but also different from each other?"

MMR is an information retrieval algorithm designed to reduce redundancy in the retrieved results while maintaining high relevance to the query.

* MMR helps especially in RAG pipeline where:
- You want your context window to contain diverse but still relevant information.
- Especially useful when documents are semantically overlapping.

* MMR Retriever avoids that by:
- Picking the most relevant document first.
- Then picking the next most relevant & least similar to already selected docs.
- And so on.---

Multi-Query Retriever
Sometimes a single query might not capture all the ways information is phrased in your documents.